

Web Application Enumeration

A Complete CTF & Pentest Cheat Sheet — Basic to Advanced

Enumeration is the process of systematically gathering every reachable piece of information about a target so that attack surface becomes visible: hosts, subdomains, technologies, endpoints, parameters, and hidden functionality. In CTFs this is almost always the phase that finds the flag path — a forgotten backup file, a hidden admin panel, or an undocumented API route. This sheet follows a logical order: run it top to bottom on a new box, or jump to the phase you need.

How to read this sheet

Each phase lists the **goal**, the **tools/commands**, and **what the output tells you**. Replace `target.com` / `TARGET_IP` with your real scope.

PHASE 0 | Scope & Ground Rules

Purpose: Confirm what you are allowed to touch before sending a single packet.

- In a CTF: scope is the box/domain/IP given by the platform (HTB, THM, CTFd, etc.).
- In real engagements: get written authorization, confirm IP ranges/domains, and note any rate-limit or blackout windows.
- Keep a running notes file (target, port, service, version, finding) — enumeration without notes is enumeration wasted.

PHASE 1 | Passive Reconnaissance (OSINT — No Direct Contact)

Purpose: Map the organization's footprint using public sources only, so the target never sees you probing yet.

1.1 Domain & Subdomain Discovery

Tool	Command	What It Reveals
Subfinder	<code>subfinder -d target.com -all -o subs.txt</code>	Passive subdomains from 20+ public sources (fast, safe first pass).
Assetfinder	<code>assetfinder --subs-only target.com</code>	Cross-check subdomains from a different data set to fill gaps.
Amass (passive)	<code>amass enum -passive -d target.com -o amass.txt</code>	Deep OSINT: certs, scraping, APIs — most thorough passive source.
crt.sh	<code>curl -s "https://crt.sh/?q=%25.target.com&output=json"</code>	Certificate Transparency logs leak subdomains issued in SSL certs.

1.2 Infrastructure & Metadata

Tool	Command	What It Reveals
WHOIS	<code>whois target.com</code>	Registrar, name servers, sometimes registrant contact/org info.
DNS records	<code>dig target.com ANY +noall +answer</code> <code>dig MX/TXT/NS target.com</code>	Mail servers, SPF/DMARC, hosting provider, third-party services in use.

Tool	Command	What It Reveals
Wayback Machine	<code>waybackurls target.com tee wayback.txt</code>	Old URLs, parameters, and endpoints that may still work but aren't linked anymore.
GitHub Dorking	<code>site:github.com "target.com" api_key</code>	Leaked credentials, internal endpoints, or config files committed by mistake.
Shodan / Censys	<code>shodan search hostname:target.com</code>	Exposed services, banners, open ports indexed from prior internet-wide scans.

Note: Passive recon should generate zero logs on the target's own servers — everything above queries third parties.

PHASE 2 | Active Reconnaissance — Hosts, Ports & Services

Purpose: Confirm which subdomains/IPs are alive, then find every open door (port) and what's answering behind it.

2.1 Live Host Filtering

Tool	Command	What It Reveals
httpx	<code>cat subs.txt httpx -silent -sc -title -ip -o live.txt</code>	Which subdomains actually resolve and respond over HTTP(S), plus status code & page title.
GoWitness / EyeWitness	<code>gowitness file -f live.txt</code>	Screenshots every live host — spot login panels, dashboards, dev/staging sites at a glance.

2.2 Port & Service Scanning

Tool	Command	What It Reveals
Nmap (fast sweep)	<code>nmap -p- --min-rate=5000 -T4 TARGET_IP -oG allports.txt</code>	Every open TCP port quickly, so you know what to scan in depth next.
Nmap (service/version)	<code>nmap -sC -sV -p 21,22,80,443,8080 TARGET_IP -oN detailed.txt</code>	Service names, version numbers, and default NSE script findings (banners, TLS certs, etc.).
Nmap NSE (web-focused)	<code>nmap --script http-enum,http-title,http-headers -p80,443 TARGET_IP</code>	HTTP-specific enumeration: common paths, server headers, page titles in one pass.
Banner grabbing	<code>nc -nv TARGET_IP 80 GET / HTTP/1.0\r\n\r\n</code>	Raw server response — reveals server software/version not shown by default scans.

Note: Always scan all 65535 ports first (-p-), not just top-1000 — CTFs love hiding services on unusual ports.

PHASE 3 | Web Technology Fingerprinting

Purpose: Identify the exact stack (CMS, framework, language, server) so you can pull matching wordlists, known CVEs, and default paths.

Tool	Command	What It Reveals
Wappalyzer / whatweb	<code>whatweb -a 3 https://target.com</code>	CMS, JS libraries, analytics, server, framework — from response headers & page markup.
HTTP headers	<code>curl -I https://target.com</code>	Server, X-Powered-By, cookies, security headers (or lack of them: CSP, HSTS).
Wafw00f	<code>wafw00f https://target.com</code>	Detects a WAF in front of the app so you know to throttle/obfuscate later payloads.
Builtwith	<code>curl -s https://target.com grep -i 'generator\ powered'</code>	Meta tags revealing CMS/generator (e.g., WordPress, Drupal, Joomla).

PHASE 4 | Virtual Host & Subdomain Brute-Forcing (Active)

Purpose: Find hosts that DNS/OSINT missed — many CTF boxes hide a second site behind a vhost header, invisible to normal browsing.

Tool	Command	What It Reveals
ffuf (vhost)	<code>ffuf -w subdomains.txt -u https://TARGET_IP -H "Host: FUZZ.target.com" -fs 4242</code>	Vhosts that respond differently in size/status than the default — reveals hidden virtual sites.
gobuster vhost	<code>gobuster vhost -u https://target.com -w subdomains.txt --append-domain</code>	Same goal as above with Gobuster's vhost mode; good cross-check.
DNS bruteforce	<code>puredns bruteforce subs-wordlist.txt target.com</code>	Active DNS guessing beyond what passive sources indexed — catches unlisted subdomains.

PHASE 5 | Content & Directory Discovery

Purpose: Enumerate files and directories that exist on the server but aren't linked anywhere in the visible site — the classic CTF goldmine.

Tool	Command	What It Reveals
Gobuster	<code>gobuster dir -u https://target.com -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak -t 50</code>	Hidden directories/files, with common backup/source extensions included.
Dirsearch	<code>dirsearch -u https://target.com -e php,html,js -t 50</code>	Similar goal with a large curated extension-aware wordlist, recursive scanning built-in.
FFUF	<code>ffuf -u https://target.com/FUZZ -w raft-large-directories.txt -mc 200,301,302,403</code>	Fast, flexible fuzzing; filter by status code to cut noise.
Feroxbuster	<code>feroxbuster -u https://target.com -w raft-large-words.txt -x php,bak,zip</code>	Recursive brute-force built for speed — auto-descends into found directories.

5.1 Sensitive File Hunting

Tool	Command	What It Reveals
Backup/source leaks	<code>ffuf -u https://target.com/FUZZ -w files.txt -e .bak,.old,.zip,.tar.gz,.swp</code>	Editor swap files, forgotten backups, old source archives — frequent CTF flag source.
Git exposure	<code>python3 git-dumper.py https://target.com/.git/ ./gitdump</code>	A leaked .git folder can rebuild the entire source history, including old secrets.
robots.txt / sitemap.xml	<code>curl https://target.com/robots.txt</code>	Developer-disclosed paths meant to be hidden from search engines — read it first, always.
.well-known / config	<code>curl https://target.com/.env</code> <code>curl https://target.com/config.php.bak</code>	Environment files/configs sometimes left readable with DB creds or API keys.

PHASE 6 | Parameter & Hidden Input Discovery

Purpose: Find GET/POST parameters the app accepts but never shows in a form — required before testing injection, IDOR, or SSRF.

Tool	Command	What It Reveals
ParamSpider	<code>paramspider -d target.com</code>	Pulls historical URLs with parameters from web archives — instant parameter wordlist.
Arjun	<code>arjun -u https://target.com/endpoint</code>	Brute-forces likely parameter names against a single endpoint and reports which are accepted.

Tool	Command	What It Reveals
FFUF (param fuzz)	<code>ffuf -u https://target.com/page?FUZZ=test -w params.txt -mc 200 -fs 1234</code>	Detects parameters by response-size delta when a param is recognized vs ignored.
x8	<code>x8 -u https://target.com/page -w params.txt</code>	Fast parameter discovery with body/query support, good for APIs too.

PHASE 7 | API & Modern App Enumeration

Purpose: Modern targets (React/Vue SPAs, mobile backends) hide most functionality behind APIs — enumerate the API surface directly.

Tool	Command	What It Reveals
JS file mining	<code>cat urls.txt grep '\.js\$' xargs -I{} curl -s {} grep -Eo "(\api\[a-zA-Z0-9_/-]+)"</code>	Endpoint paths, sometimes hardcoded keys/tokens, hidden inside bundled JavaScript.
LinkFinder	<code>python3 linkfinder.py -i https://target.com/app.js -o cli</code>	Automates extracting endpoint paths and parameters from minified JS.
Swagger/OpenAPI probe	<code>curl https://target.com/swagger.json curl https://target.com/api/docs</code>	If exposed, gives the full documented API surface — an enumeration shortcut.
GraphQL introspection	<code>curl -X POST -d '{"query":"{__schema{types{name}}}}"' https://target.com/graphql</code>	Introspection (if enabled) reveals the entire schema: queries, mutations, types.
Postman/Insomnia leaks	<code>site:github.com filename:*.postman_collection.json target</code>	Leaked collections sometimes document private/internal API routes.

PHASE 8 | Authentication & Session Enumeration

Purpose: Map how identity works in the app — often the fastest path to privilege escalation or account takeover in a CTF.

Tool	Command	What It Reveals
Login endpoint behavior	Test valid vs invalid username — compare response time/message	User enumeration via different error messages or timing differences.
Cookie/JWT inspection	<code>echo cut -d. -f2 base64 -d</code>	JWT header/payload in plaintext — algorithm used, claims, expiry, sometimes role/user id.
Password reset flow	Manually walk the reset flow, inspect every request/response	Predictable tokens, IDOR on reset (resetting another user's password), verbose responses.
Default creds	<code>hydra -L users.txt -P rockyou.txt target.com http-post-form "/login:user=^USER^&pass;=^PASS^:F=incorrect"</code>	Weak/default credentials — common in intentionally-vulnerable CTF boxes.

Note: Throttle or avoid brute force on real client engagements without explicit written permission — lockouts and legal exposure are real.

PHASE 9 | CMS-Specific Enumeration

Purpose: Once a CMS is fingerprinted (Phase 3), switch to purpose-built tools that know its exact folder structure and plugin ecosystem.

Tool	Command	What It Reveals
WordPress	<code>wpscan --url https://target.com --enumerate vp,vt,u</code>	Vulnerable plugins/themes, valid usernames — WordPress-specific weak points.
Joomla	<code>joomscan -u https://target.com</code>	Joomla version, known vulnerable components, exposed admin paths.
Drupal	<code>droopescan scan drupal -u https://target.com</code>	Drupal version and module fingerprinting against known CVEs.
Generic CMS paths	<code>ffuf -u https://target.com/FUZZ -w cms-wordlist.txt</code>	<code>wp-admin</code> , <code>/administrator</code> , <code>/user/login</code> — CMS-specific admin entry points.

PHASE 10 | Advanced & Edge-Case Techniques

Purpose: For when the basics don't find the flag — dig into infrastructure quirks, cloud misconfig, and app logic.

Tool	Command	What It Reveals
Cloud storage buckets	<code>aws s3 ls s3://target-assets --no-sign-request</code>	Publicly readable S3/GCS/Azure buckets sometimes hold backups, source, or credentials.
Subdomain takeover check	<code>subjack -w subs.txt -t 50 -o takeovers.txt</code>	Dangling CNAMEs pointing to unclaimed cloud services — a full host compromise if taken over.
HTTP request smuggling probe	<code>smuggler.py -u https://target.com</code>	Front-end/back-end parsing mismatches, relevant behind reverse proxies/load balancers.
CORS misconfig check	<code>curl -H "Origin: https://evil.com" -I https://target.com/api/data</code>	Overly permissive Access-Control-Allow-Origin/Credentials headers enabling cross-site data theft.
Cache deception/poisoning	<code>curl -I https://target.com/account.php/nonexistent.css</code>	Whether static-file caching rules can be tricked into caching authenticated/private pages.
Rate-limit / logic fuzz	<code>ffuf -u https://target.com/api/FUZZ -w numeric-ids.txt</code>	IDOR: incrementing IDs in API paths that return other users' data without authorization checks.

Note: Advanced techniques above are enumeration/detection steps only — exploitation of any finding must stay inside your authorized CTF/engagement scope.

PHASE 11 | Putting It Together — A Repeatable Workflow

Purpose: A sane default order to run on any new web target, CTF or real engagement.

- 1. Passive OSINT (Phase 1) → build a subdomain/asset list without touching the target.

- 2. Resolve live hosts + screenshot them (Phase 2.1) → prioritize what to look at first.
- 3. Full port scan + service versions (Phase 2.2) → know every entry point, not just port 80/443.
- 4. Fingerprint tech stack (Phase 3) → pick the right wordlists/tools for the next steps.
- 5. Vhost + content + parameter discovery (Phases 4–6) → map the full attack surface.
- 6. API/JS mining and auth mapping (Phases 7–8) → find the app's real functionality.
- 7. CMS-specific and advanced checks (Phases 9–10) → squeeze out anything the generic pass missed.
- 8. Document everything as you go — a flag is only useful if you can explain how you found it.

Prepared as a personal CTF/pentest reference. Always operate within authorized scope.